

Funkce v C++

Příprava k maturitě - SPŠT IT

Obsah

1 Pojem funkce	2
2 Definice funkce	2
2.1 Syntaxe	2
2.2 Části funkce	2
2.3 Příklady	2
3 Umístění definice funkce	4
3.1 Před funkcí main()	4
3.2 Po funkci main() s prototypem	4
3.3 Hlavičkové soubory	4
4 Volání funkce	5
4.1 Formální parametry	5
4.2 Skutečné parametry (argumenty)	6
5 Lokální a globální proměnné	6
5.1 Lokální proměnné	6
5.2 Globální proměnné	6
6 Typy parametrů funkcí	7
6.1 Předávání hodnotou (by value)	7
6.2 Předávání referencí (by reference)	7
6.3 Předávání ukazatelem (by pointer)	8
6.4 Konstantní parametry	8
6.5 Výchozí hodnoty parametrů	9
7 Přetěžování funkcí (Function Overloading)	9
7.1 Pravidla přetěžování	9
7.2 Příklady přetěžování	9
7.3 Praktický příklad – tisk různých typů	10
7.4 Výhody přetěžování	10
Odpovědi na zadané podotázky	11
Další materiály a zdroje	11

1 Pojem funkce

Funkce je pojmenovaný blok kódu, který provádí určitou úlohu. Funkce umožňují:

- **Znovupoužitelnost kódu** – jednou napsaný kód lze volat vícekrát
- **Modularitu** – rozdělení programu na menší, logické celky
- **Čitelnost** – přehlednější a srozumitelnější kód
- **Údržbu** – snadnější opravy a změny
- **Abstrakci** – skrytí implementačních detailů

2 Definice funkce

Funkce v C++ má dvě části: **deklaraci** (prototyp) a **implementaci** (tělo funkce).

2.1 Syntaxe

cpp

```
1 // Deklarace (prototyp) – říká překladači, že funkce existuje
2 navratovyTyp nazevFunkce(typParametru1 parametr1, typParametru2 parametr2);
3
4 // Implementace (definice) – obsahuje kód funkce
5 navratovyTyp nazevFunkce(typParametru1 parametr1, typParametru2 parametr2) {
6     // tělo funkce
7     return hodnota; // pokud návratový typ není void
8 }
```

2.2 Části funkce

1. **Návratový typ** – typ hodnoty, kterou funkce vrátí (int, double, void...)
2. **Název funkce** – identifikátor funkce
3. **Seznam parametrů** – vstupy funkce (může být prázdný)
4. **Tělo funkce** – kód ohraničený složenými závorkami {}
5. **Příkaz return** – vrátí hodnotu (kromě funkcí typu void)

2.3 Příklady

cpp

```
1 // Funkce bez parametrů a bez návratové hodnoty
2 void pozdrav() {
3     cout << "Ahoj!" << endl;
4 }
5
6 // Funkce s parametry a návratovou hodnotou
7 int soucet(int a, int b) {
8     return a + b;
9 }
10
11 // Funkce s více parametry
12 double objem(double delka, double sirka, double vyska) {
13     return delka * sirka * vyska;
14 }
15
```

```
16 // Funkce vracející boolean
17 bool jeKladne(int cislo) {
18     return cislo > 0;
19 }
```

3 Umístění definice funkce

3.1 Před funkcí main()

cpp

```

1 #include <iostream>
2 using namespace std;
3
4 // Funkce definována před main() – není potřeba prototyp
5 int soucet(int a, int b) {
6     return a + b;
7 }
8
9 int main() {
10     int vysledek = soucet(5, 3);
11     cout << vysledek << endl;
12     return 0;
13 }
```

3.2 Po funkci main() s prototypem

cpp

```

1 #include <iostream>
2 using namespace std;
3
4 // Prototyp (deklarace) před main()
5 int soucet(int a, int b);
6
7 int main() {
8     int vysledek = soucet(5, 3);
9     cout << vysledek << endl;
10    return 0;
11 }
12
13 // Implementace po main()
14 int soucet(int a, int b) {
15     return a + b;
16 }
```

3.3 Hlavičkové soubory

Pro větší projekty se funkce rozdělují do samostatných souborů:

matematika.h (hlavičkový soubor – deklarace):

cpp

```

1 #ifndef MATEMATIKA_H
2 #define MATEMATIKA_H
3
4 int soucet(int a, int b);
5 int rozdil(int a, int b);
6 double prumer(int a, int b);
7
8 #endif
```

matematika.cpp (zdrojový soubor – implementace):

cpp

```

1 #include "matematika.h"
2
3 int soucet(int a, int b) {
4     return a + b;
5 }
6
7 int rozdil(int a, int b) {
8     return a - b;
9 }
10
11 double prumer(int a, int b) {
12     return (a + b) / 2.0;
13 }

```

main.cpp (hlavní program):

cpp

```

1 #include <iostream>
2 #include "matematika.h"
3 using namespace std;
4
5 int main() {
6     cout << soucet(10, 5) << endl;    // 15
7     cout << rozdil(10, 5) << endl;    // 5
8     cout << prumer(10, 5) << endl;    // 7.5
9     return 0;
10 }

```

4 Volání funkce

Volání funkce znamená provedení (spuštění) kódu, který je v těle funkce definován.

cpp

```

1 int soucet(int a, int b) {    // a, b jsou FORMÁLNÍ parametry
2     return a + b;
3 }
4
5 int main() {
6     int x = 10, y = 20;
7     int vysledek = soucet(x, y);    // x, y jsou SKUTEČNÉ parametry
8     // nebo přímo:
9     int vysledek2 = soucet(5, 8);    // 5, 8 jsou SKUTEČNÉ parametry
10    return 0;
11 }

```

4.1 Formální parametry

- Parametry v **definici** funkce
- Specifikují typ a název vstupních hodnot
- Existují pouze během provádění funkce
- Fungují jako lokální proměnné

4.2 Skutečné parametry (argumenty)

- Hodnoty předané při **volání** funkce
- Mohou být: konstanty, proměnné, výrazy
- Musí odpovídat typem formálním parametrům
- Počet a pořadí musí souhlasit

cpp

```

1 double mocnina(double zaklad, int exponent) {
2     double vysledek = 1;
3     for (int i = 0; i < exponent; i++) {
4         vysledek *= zaklad;
5     }
6     return vysledek;
7 }
8
9 int main() {
10    // Různé způsoby volání
11    double a = mocnina(2.0, 3);           // s proměnnými
12    double b = mocnina(5, 2);             // s konstantami
13    double c = mocnina(1.5, 2 + 2);       // s výrazem
14    return 0;
15 }
```

5 Lokální a globální proměnné

5.1 Lokální proměnné

- Deklarované **uvnitř** funkce nebo bloku
- Existují pouze během provádění funkce
- Každé volání funkce vytváří novou instanci
- Po ukončení funkce jsou zničeny
- Mají přednost před globálními proměnnými se stejným názvem

cpp

```

1 void funkce() {
2     int lokalniPromenna = 10; // lokální proměnná
3     // použitelná jen v této funkci
4 }
5
6 int main() {
7     int x = 5; // lokální v main()
8     funkce();
9     // zde nelze použít lokalniPromenna ani x z funkce()
10    return 0;
11 }
```

5.2 Globální proměnné

- Deklarované **mimo** všechny funkce
- Dostupné ve všech funkcích v programu
- Existují po celou dobu běhu programu
- Uchovávají hodnotu mezi voláními funkcí

- **Nedoporučuje se nadužívat** – horší čitelnost a údržba

cpp

```

1 #include <iostream>
2 using namespace std;
3
4 int globalniPromenna = 100; // globální proměnná
5
6 void funke1() {
7     cout << globalniPromenna << endl; // přístup k globální
8     globalniPromenna = 200;           // změna globální
9 }
10
11 void funke2() {
12     int globalniPromenna = 50; // lokální – zastíní globální
13     cout << globalniPromenna << endl; // vypíše 50
14     cout << ::globalniPromenna << endl; // :: přístup ke globální
15 }
16
17 int main() {
18     funke1(); // vypíše 100, změní na 200
19     funke1(); // vypíše 200
20     funke2(); // vypíše 50, pak 200
21     return 0;
22 }

```

6 Typy parametrů funkcí

6.1 Předávání hodnotou (by value)

- Standardní způsob předávání parametrů
- Vytvoří se **kopie** skutečného parametru
- Změny uvnitř funkce neovlivní původní proměnnou
- Vhodné pro malé datové typy

cpp

```

1 void zmena(int x) {
2     x = 100; // změní pouze lokální kopii
3 }
4
5 int main() {
6     int a = 5;
7     zmena(a);
8     cout << a << endl; // vypíše 5 (původní hodnota)
9     return 0;
10 }

```

6.2 Předávání referencí (by reference)

- Používá se symbol &
- Nepředává se kopie, ale **odkaz** na originální proměnnou
- Změny uvnitř funkce **ovlivní** původní proměnnou
- Efektivnější pro velké datové struktury

- Umožňuje vracet více hodnot

cpp

```

1 void zmena(int &x) { // & označuje referenci
2     x = 100; // změní původní proměnnou
3 }
4
5 void prohod(int &a, int &b) {
6     int temp = a;
7     a = b;
8     b = temp;
9 }
10
11 int main() {
12     int a = 5;
13     zmena(a);
14     cout << a << endl; // vypíše 100 (změněná hodnota)
15
16     int x = 10, y = 20;
17     prohod(x, y);
18     cout << x << ", " << y << endl; // vypíše 20, 10
19     return 0;
20 }

```

6.3 Předávání ukazatelem (by pointer)

- Používá se symbol * pro typ a & pro získání adresy
- Předává se **adresa** proměnné v paměti
- Funkce pracuje s adresou pomocí operátoru dereference *
- Starší způsob, reference jsou modernější

cpp

```

1 void zmena(int *x) { // ukazatel na int
2     *x = 100; // změna hodnoty na adrese
3 }
4
5 int main() {
6     int a = 5;
7     zmena(&a); // předání adresy proměnné a
8     cout << a << endl; // vypíše 100
9     return 0;
10 }

```

6.4 Konstantní parametry

- Používá se klíčové slovo `const`
- Zabrání změně hodnoty parametru
- Užitečné pro velké objekty předávané referencí

cpp

```

1 void vypis(const int &x) { // konstantní reference
2     cout << x << endl;
3     // x = 10; // CHYBA – nelze měnit const parametr
4 }
5
6 double obvod(const double pi, const double r) {

```



```

7     return 2 * pi * r;
8     // pi = 3;    // CHYBA – nelze měnit
9 }

```

6.5 Výchozí hodnoty parametrů

- Parametry mohou mít výchozí hodnoty
- Výchozí hodnoty se zadávají v deklaraci (prototypu)
- Parametry s výchozí hodnotou musí být na konci seznamu

cpp

```

1 // V prototypu nebo definici
2 void tiskni(string text, int pocet = 1) {
3     for (int i = 0; i < pocet; i++) {
4         cout << text << endl;
5     }
6 }
7
8 int main() {
9     tiskni("Ahoj");           // použije výchozí pocet = 1
10    tiskni("Ahoj", 3);        // použije pocet = 3
11    return 0;
12 }

```

7 Přetěžování funkcí (Function Overloading)

Přetěžování funkcí umožňuje definovat více funkcí se stejným názvem, ale s různými parametry.

7.1 Pravidla přetěžování

Funkce se liší:

1. Počtem parametrů
2. Typy parametrů
3. Pořadím typů parametrů

Funkce se **neliší** pouze návratovým typem!

7.2 Příklady přetěžování

cpp

```

1 // Různý počet parametrů
2 int soucet(int a, int b) {
3     return a + b;
4 }
5
6 int soucet(int a, int b, int c) {
7     return a + b + c;
8 }
9
10 // Různé typy parametrů
11 double soucet(double a, double b) {

```

```

12     return a + b;
13 }
14
15 string soucet(string a, string b) {
16     return a + b; // spojení řetězců
17 }
18
19 // Použití
20 int main() {
21     cout << soucet(5, 3) << endl;           // volá int, int
22     cout << soucet(5, 3, 2) << endl;         // volá int, int, int
23     cout << soucet(2.5, 3.7) << endl;       // volá double, double
24     cout << soucet("Ahoj ", "svete") << endl; // volá string, string
25     return 0;
26 }

```

7.3 Praktický příklad – tisk různých typů

cpp

```

1  #include <iostream>
2  using namespace std;
3
4  void vypis(int x) {
5      cout << "Celé číslo: " << x << endl;
6  }
7
8  void vypis(double x) {
9      cout << "Reálné číslo: " << x << endl;
10 }
11
12 void vypis(string x) {
13     cout << "Text: " << x << endl;
14 }
15
16 void vypis(int x, int y) {
17     cout << "Dva integrity: " << x << ", " << y << endl;
18 }
19
20 int main() {
21     vypis(42);           // volá vypis(int)
22     vypis(3.14);         // volá vypis(double)
23     vypis("Hello");      // volá vypis(string)
24     vypis(10, 20);       // volá vypis(int, int)
25     return 0;
26 }

```

7.4 Výhody přetěžování

1. Jednoduché rozhraní – stejný název pro podobné operace
2. Čitelnější kód – není nutné vymýšlet různé názvy
3. Intuitivní použití – programátor nemusí pamatovat různé názvy
4. Polymorfismus – základ objektově orientovaného programování

Odpovědi na zadané podotázky

Každá otázka na začátku odpovědi obsahuje odkaz na konkrétní kapitolu v zápise s proklikem

Další materiály a zdroje

Žádné další zdroje
